BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT HƯNG YÊN

BÀI TẬP LỚN

ỨNG DỤNG HỌC MÁY DỰ ĐOÁN TÌNH TRẠNG CÂN NẶNG

MÔN HỌC: HỌC MÁY CƠ BẢN

NGÀNH: KỸ THUẬT PHẦN MỀM

CHUYÊN NGÀNH: PHÁT TRIỂN ỨNG DỤNG PHẦN MỀM

SINH VIÊN: **TRẦN MAI LAN - 10123196**

LÊ TIẾN LINH - 12523040

MÃ LỚP: **12523W.4**

HƯỚNG DẪN: **TS. Hoàng Quốc Việt**

HƯNG YÊN – 2026

NHẬN XÉT

Nhận xét của giảng viên hướng dẫn:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

LỜI CAM ĐOAN

Nhóm em xin cam đoan bài tập lớn "Ứng dụng học máy dự đoán tình trạng cân nặng" là kết quả thực hiện của bản thân em dưới sự hướng dẫn của TS. Hoàng Quốc Việt.

Những phần sử dụng tài liệu tham khảo trong bài tập lớn đã được nêu rõ trong phần tài liệu tham khảo. Các kết quả trình bày trong bài tập lớn và chương trình xây dựng được hoàn toàn là kết quả do cả nhóm thực hiện.

Nếu vi phạm lời cam đoan này, em xin chịu hoàn toàn trách nhiệm trước khoa và nhà trường.

Hưng Yên, ngày ... tháng ... năm 2026

SINH VIÊN 1	**SINH VIÊN 2**
(Ký, ghi rõ họ tên)	*(Ký, ghi rõ họ tên)*

LỜI CẢM ƠN

Để có thể hoàn thành bài tập lớn này, lời đầu tiên nhóm em xin phép gửi lời cảm ơn tới bộ môn Công nghệ phần mềm, Khoa Công nghệ thông tin – Trường Đại học Công nghệ Kỹ thuật Hưng Yên đã tạo điều kiện thuận lợi cho em thực hiện bài tập lớn môn học này.

Đặc biệt nhóm em xin chân thành cảm ơn TS. Hoàng Quốc Việt đã rất tận tình hướng dẫn, chỉ bảo em trong suốt thời gian thực hiện bài tập lớn vừa qua.

Nhóm em cũng xin chân thành cảm ơn tất cả các thầy/cô trong trường đã tận tình giảng dạy, trang bị cho em những kiến thức cần thiết, quý báu để giúp em thực hiện được bài tập lớn này.

Mặc dù nhóm em đã có cố gắng, nhưng với trình độ còn hạn chế, trong quá trình thực hiện đề tài không tránh khỏi những thiếu sót. Em hi vọng sẽ nhận được những ý kiến nhận xét, góp ý của các thầy/cô về những kết quả triển khai trong bài tập lớn.

Em xin trân trọng cảm ơn!

MỤC LỤC

NHẬN XÉT 2
DANH MỤC CÁC THUẬT NGỮ 7
DANH MỤC CÁC BẢNG 8
DANH MỤC CÁC HÌNH VẼ 9
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 11
1.1. Lý do chọn đề tài 11
1.2. Trình bày dữ liệu bài toán 12
1.3. Tiền xử lý dữ liệu 14
1.4. Làm sạch dữ liệu 14
1.5. Trực quan hoá dữ liệu 14
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 16
2.1. Mạng nơ-ron nhân tạo nhiều lớp 16
2.1.1. Giới thiệu 16
2.1.2. Cấu trúc mô hình 16
2.1.3. Quá trình xây dựng mô hình 16
2.1.4. Ưu điểm 16
2.1.5. Nhược điểm 17
2.1.6. Đánh giá hiệu suất 17
2.2. Các kỹ thuật tiền xử lý 17
2.2.1. Mã hoá One-Hot 17
2.2.2. Chuẩn hoá StandardScaler 17
CHƯƠNG 3: GIẢI PHÁP, CÀI ĐẶT THỰC HIỆN 18
3.1 Mã nguồn tiền xử lý dữ liệu 18
3.2 Mã nguồn chức năng làm sạch dữ liệu 19
3.2.1. Xử lý giá trị dữ liệu thiếu 19
3.2.2. Kiểm tra dòng trùng lặp 19
3.2.3. Kiểm tra ngoại lai 20
3.2.4. Ma trận tương quan 21
3.3 Mã nguồn chức năng trực quan hóa dữ liệu 22
3.3.1. Phân phối 7 mức cân nặng 22

3.3.2. Phân phối các đặc trưng liên tục 23
3.3.3. Quan hệ giữa thói quen và mức cân nặng 24
3.3.4. Phân bố Weight theo từng mức cân nặng 28
3.4. Chuyển đổi dữ liệu 28
3.4.1. Mã hoá nhãn và xử lý dữ liệu chuỗi 28
3.4.2. Chia dữ liệu và chuẩn hoá 29
3.4.3. Về dữ liệu mất cân bằng 30
3.5. Huấn luyện mô hình 30
3.5.1. Xây dựng mạng nơ-ron 30
3.5.2. Huấn luyện và xuất file .h5 31
3.5.3. Đánh giá mô hình 33
CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG 35
KẾT LUẬN 39
TÀI LIỆU THAM KHẢO 41

DANH MỤC CÁC THUẬT NGỮ

STT	Từ viết tắt	Cụm từ tiếng Anh	Diễn giải
1	ANN	Artificial Neural Network	Mạng nơ-ron nhân tạo
2	MLP	Multi-Layer Perceptron	Mạng nơ-ron truyền thẳng nhiều lớp
3	BMI	Body Mass Index	Chỉ số khối cơ thể (cân nặng / chiều cao²)
4	ReLU	Rectified Linear Unit	Hàm kích hoạt \(f(x) = \max(0, x)\)
5	SMOTE	Synthetic Minority Over-sampling Technique	Kỹ thuật sinh thêm mẫu cho lớp ít dữ liệu
6	IQR	Interquartile Range	Khoảng tứ phân vị (Q3 − Q1)
7	KDE	Kernel Density Estimation	Ước lượng hàm mật độ, đường cong trên histogram
8	SVM	Support Vector Machine	Máy vector hỗ trợ
9	SHAP	SHapley Additive exPlanations	Phương pháp giải thích đóng góp của từng đặc trưng
10	LIME	Local Interpretable Model-agnostic Explanations	Phương pháp giải thích dự đoán cục bộ
11	KB	Kilobyte	Đơn vị dung lượng (nằm ở phần cấu trúc mô hình)
12	TV	Television	Ti vi (trong câu “điện thoại/máy tính/TV”)

DANH MỤC CÁC BẢNG

Bảng 1.1. Phân bổ nhãn *14*

DANH MỤC CÁC HÌNH VẼ

Hình 1.1. Hiển thị 5 hàng đầu của dữ liệu 12
Hình 1.2. Thông tin tổng quan về dữ liệu 12
Hình 1.3. Mô tả dữ liệu 13
Hình 3.1. Import thư viện, đọc và hiển thị 5 hàng đầu của DataFrame 18
Hình 3.2. Kết quả hiển thị 5 hàng đầu của DataFrame 18
Hình 3.3. Mã nguồn xử lý giá trị thiếu 19
Hình 3.4. Hiển thị phần trăm giá trị thiếu của từng cột 19
Hình 3.5. Mã nguồn hiển thị tổng số dòng trùng lặp trong dữ liệu 19
Hình 3.6. Kết quả của số dòng trùng 20
Hình 3.7. Mã nguồn kiểm tra ngoại lai 20
Hình 3.8. Kết quả kiểm tra ngoại lai 21
Hình 3.9. Heatmap tương quan giữa các đặc trưng số 21
Hình 3.10 Kết quả ma trận tương quan 22
Hình 3.11. Mã nguồn phân phối 7 mức cân nặng 22
Hình 3.12. Kết quả phân phối 7 mức cân nặng 23
Hình 3.13. Mã nguồn phân phối các đặc trưng liên tục 23
Hình 3.14. Kết quả phân phối các đặc trưng liên tục 24
Hình 3.15. Mã nguồn quan hệ giữa thói quen và mức cân nặng 24
Hình 3.16. Quan hệ giữa Gender và mức cân nặng 25
Hình 3.17. Quan hệ giữa FAVC và mức cân nặng 25
Hình 3.18. Quan hệ giữa FAF và mức cân nặng 26
Hình 3.19. Quan hệ giữa CH2O và mức cân nặng 26
Hình 3.20. Quan hệ giữa MTRANS và mức cân nặng 27
Hình 3.21. Quan hệ giữa CAEC và mức cân nặng 27
Hình 3.22. Mã nguồn phân bố Weight theo từng hạng cân 28
Hình 3.23. Mức cân nặng theo từng hạng cân 28
Hình 3.24. Mã nguồn mã hoá nhãn và xử lý dữ liệu chuỗi 29
Hình 3.25. Mã hoá nhãn và xử lý dữ liệu chuỗi 29
Hình 3.26. Mã nguồn chia dữ liệu và chuẩn hoá 30
Hình 3.27. Kết quả chia dữ liệu và chuẩn hoá 30
Hình 3.28. Mã nguồn xây dựng mạng Nơ ron 31
Hình 3.29. Cấu trúc mô hình 31
Hình 3.30. Mã nguồn huấn luyện và xuất file .h5 32

Hình 3.31. Mã nguồn vẽ đồ thị loss và accuracy theo epoch ***32***
Hình 3.32. Đồ thị Loss và Accuracy ***33***
Hình 3.33. Mã nguồn Đánh giá mô hình ***33***
Hình 3.34 Kết quả đánh giá mô hình ***34***
Hình 4.1. Cấu trúc thư mục ***35***
Hình 4.2. Trang giới thiệu ***36***
Hình 4.3. Trang nhập thông tin ***37***
Hình 4.4. Kết quả dự đoán ***38***

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong xã hội hiện đại, thói quen ăn uống và sinh hoạt của con người đang có nhiều thay đổi. Việc sử dụng thức ăn nhanh, đồ uống nhiều đường, ít vận động và dành nhiều thời gian trước màn hình có thể ảnh hưởng đến tình trạng cân nặng. Bên cạnh thừa cân và béo phì, thiếu cân cũng là vấn đề cần được quan tâm. Vì vậy, việc theo dõi tình trạng cân nặng và xây dựng lối sống lành mạnh ngày càng trở nên cần thiết.

Hiện nay, nhiều người đánh giá tình trạng cân nặng thông qua cảm nhận cá nhân hoặc chỉ số khối cơ thể BMI. Tuy nhiên, việc tự đánh giá bằng cảm nhận có thể thiếu khách quan, trong khi BMI chủ yếu phản ánh mối quan hệ giữa chiều cao và cân nặng, không trực tiếp thể hiện thói quen ăn uống, vận động và sinh hoạt của mỗi người. Bên cạnh đó, các phương pháp thông thường chưa cung cấp công cụ dự đoán dựa trên dữ liệu và hiển thị xác suất tương ứng với từng mức cân nặng để người dùng tham khảo.

Trong khi đó, sự phát triển của khoa học dữ liệu và học máy mở ra khả năng phân tích dữ liệu, nhận diện các mối liên hệ và hỗ trợ dự đoán. Việc ứng dụng học máy vào bài toán phân loại tình trạng cân nặng có thể cung cấp thêm thông tin tham khảo, giúp người dùng quan tâm hơn đến thói quen sinh hoạt của mình.

Xuất phát từ những lý do trên, nhóm em lựa chọn đề tài "Ứng dụng học máy dự đoán tình trạng cân nặng" nhằm xây dựng mô hình dự đoán tình trạng cân nặng dựa trên các đặc trưng về thói quen ăn uống, vận động và sinh hoạt. Đây là bài toán phân loại đa lớp (Multi-class Classification), trong đó đầu ra của mô hình gồm 7 mức: thiếu cân, cân nặng bình thường, thừa cân độ I, thừa cân độ II, béo phì độ I, béo phì độ II và béo phì độ III.

Để giải quyết bài toán, em sử dụng mạng nơ-ron nhân tạo được xây dựng bằng thư viện Keras, sau đó tích hợp mô hình vào website sử dụng Flask. Hệ thống cho phép người dùng nhập thông tin thông qua biểu mẫu, nhận kết quả dự đoán và theo dõi xác suất của từng mức cân nặng qua biểu đồ thanh trực quan.

Thông qua đề tài, em mong muốn xây dựng một công cụ hỗ trợ dự đoán dễ sử dụng, góp phần nâng cao nhận thức về mối liên hệ giữa thói quen sinh hoạt và tình

trạng cân nặng. Tuy nhiên, kết quả chỉ mang tính tham khảo, không thay thế việc đánh giá sức khỏe hoặc tư vấn của bác sĩ và chuyên gia dinh dưỡng.

1.2. Trình bày dữ liệu bài toán

Dữ liệu sử dụng là file *obesity_level.csv*, gồm 20.758 dòng × 17 cột, mỗi dòng là một người với các thông tin về thói quen sống và nhãn mức cân nặng. Dữ liệu không có giá trị thiếu.

	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	FCVC	NCP	CAEC	SMOKE	CH2O	SCC	FAF	TUE	CALC	MTRANS	NObeyesdad
0	Male	24.443011	1.699998	81.669950	1	1	2	3	Sometimes	0	3	0	0	1	Sometimes	Public_Transportation	Overweight_Level_II
1	Female	18.000000	1.560000	57.000000	1	1	2	3	Frequently	0	2	0	1	1	no	Automobile	Normal_Weight
2	Female	18.000000	1.711460	50.165754	1	1	2	1	Sometimes	0	2	0	1	2	no	Public_Transportation	Insufficient_Weight
3	Female	20.952737	1.710730	131.274851	1	1	3	3	Sometimes	0	2	0	1	1	Sometimes	Public_Transportation	Obesity_Type_III
4	Male	31.641081	1.914186	93.798055	1	1	3	2	Sometimes	0	2	0	2	1	Sometimes	Public_Transportation	Overweight_Level_II

Hình 1.1. Hiển thị 5 hàng đầu của dữ liệu

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20758 entries, 0 to 20757
Data columns (total 17 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Gender                          20758 non-null  object
 1   Age                             20758 non-null  float64
 2   Height                          20758 non-null  float64
 3   Weight                          20758 non-null  float64
 4   family_history_with_overweight  20758 non-null  int64
 5   FAVC                            20758 non-null  int64
 6   FCVC                            20758 non-null  int64
 7   NCP                             20758 non-null  int64
 8   CAEC                            20758 non-null  object
 9   SMOKE                           20758 non-null  int64
 10  CH2O                            20758 non-null  int64
 11  SCC                             20758 non-null  int64
 12  FAF                             20758 non-null  int64
 13  TUE                             20758 non-null  int64
 14  CALC                            20758 non-null  object
 15  MTRANS                          20758 non-null  object
 16  NObeyesdad                      20758 non-null  object
dtypes: float64(3), int64(9), object(5)
memory usage: 2.7+ MB
```

Hình 1.2. Thông tin tổng quan về dữ liệu

- Ý nghĩa các cột:
  1. Gender: Giới tính (Male/Female).
  2. Age: Tuổi (năm).
  3. Height: Chiều cao (mét).

4. Weight: Cân nặng (kg).
5. family_history_with_overweight: Gia đình có người thừa cân (0: Không, 1: Có).
6. FAVC: Thường xuyên ăn đồ nhiều calo (0/1).
7. FCVC: Mức độ ăn rau trong bữa ăn (1: Không bao giờ, 2: Thỉnh thoảng, 3: Luôn luôn).
8. NCP: Số bữa chính mỗi ngày (1 đến 4).
9. CAEC: Ăn vặt giữa các bữa chính (Không bao giờ, Thỉnh thoảng, Thường xuyên, Luôn luôn).
10. SMOKE: Hút thuốc lá (0/1).
11. CH2O: Lượng nước uống mỗi ngày (1: dưới 1 lít, 2: 1 đến 2 lít, 3: trên 2 lít).
12. SCC: Có theo dõi lượng calo ăn vào (0/1).
13. FAF: Tần suất vận động (0: không, 1: 1 đến 2 ngày/tuần, 2: 3 đến 4 ngày/tuần, 3: từ 5 ngày/tuần).
14. TUE: Thời gian dùng thiết bị điện tử (0: 0 đến 2 giờ, 1: 3 đến 5 giờ, 2: trên 5 giờ).
15. CALC: Mức độ uống rượu bia (Không bao giờ, Thỉnh thoảng, Thường xuyên).
16. MTRANS: Phương tiện di chuyển chính (Phương tiện công cộng, Đi bộ, Ô tô, Xe máy, Xe đạp).
17. NObeyesdad: Nhãn cần dự đoán, gồm 7 mức cân nặng.

- Sau khi mô tả dữ liệu ta có:

	Age	Height	Weight	family_history_with_overweight	FAVC	FCVC	NCP	SMOKE	CH2O	SCC	FAF	TUE
count	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000	20758.000000
mean	23.841804	1.700245	87.887768	0.819636	0.914443	2.449802	2.764958	0.011803	2.054581	0.033096	0.985548	0.624771
std	5.688072	0.087312	26.379443	0.384500	0.279716	0.565896	0.722398	0.108000	0.681172	0.178891	0.867857	0.655701
min	14.000000	1.450000	39.000000	0.000000	0.000000	1.000000	1.000000	0.000000	1.000000	0.000000	0.000000	0.000000
25%	20.000000	1.631856	66.000000	1.000000	1.000000	2.000000	3.000000	0.000000	2.000000	0.000000	0.000000	0.000000
50%	22.815416	1.700000	84.064875	1.000000	1.000000	2.000000	3.000000	0.000000	2.000000	0.000000	1.000000	1.000000
75%	26.000000	1.762887	111.600553	1.000000	1.000000	3.000000	3.000000	0.000000	3.000000	0.000000	2.000000	1.000000
max	61.000000	1.975663	165.057269	1.000000	1.000000	3.000000	4.000000	1.000000	3.000000	1.000000	3.000000	2.000000

Hình 1.3. Mô tả dữ liệu

+ Tuổi nhỏ nhất là 14, lớn nhất là 61, trung bình khoảng 23,8.
+ Chiều cao từ 1,45 m đến 1,98 m, trung bình khoảng 1,70 m.
+ Cân nặng từ 39 kg đến 165,06 kg, trung bình khoảng 87,9 kg.

+ Khoảng 82% người có gia đình từng thừa cân; khoảng 91,4% thường xuyên ăn đồ nhiều calo.
+ Chỉ khoảng 1,2% hút thuốc và 3,3% có theo dõi calo.

- Phân bố nhãn khá cân bằng:

Bảng 1.1. Phân bổ nhãn

Mức cân nặng	Số mẫu	Tỷ lệ
Obesity_Type_III	4.046	19,5%
Obesity_Type_II	3.248	15,6%
Normal_Weight	3.082	14,8%
Obesity_Type_I	2.910	14,0%
Insufficient_Weight	2.523	12,2%
Overweight_Level_II	2.522	12,1%
Overweight_Level_I	2.427	11,7%

1.3. Tiền xử lý dữ liệu

- Đọc dữ liệu và hiển thị 5 hàng đầu.
- Hiển thị số hàng, số cột của dữ liệu.
- Hiển thị thông tin và mô tả thống kê của dữ liệu.
- Hiển thị giá trị duy nhất của các cột chữ (Gender, CAEC, CALC, MTRANS, NObeyesdad).
- Tách đặc trưng X (16 cột đầu) và nhãn y (cột cuối).

1.4. Làm sạch dữ liệu

- Kiểm tra giá trị thiếu: không có cột nào thiếu dữ liệu.
- Kiểm tra dòng trùng lặp: có 301 dòng trùng (khoảng 1,45%).
- Kiểm tra giá trị ngoại lai bằng boxplot.
- Ma trận tương quan giữa các cột số.

1.5. Trực quan hoá dữ liệu

- Biểu đồ phân phối 7 mức cân nặng.
- Biểu đồ phân phối các đặc trưng liên tục (Age, Height, Weight).

- Biểu đồ quan hệ giữa các thói quen (FAVC, FAF, CH2O, MTRANS...) và mức cân nặng.
- Biểu đồ boxplot kiểm tra giá trị ngoại lai.
- Biểu đồ heatmap ma trận tương quan.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Mạng nơ-ron nhân tạo nhiều lớp

2.1.1. Giới thiệu

Mạng nơ-ron nhân tạo (Artificial Neural Networks – ANN) là một mô hình tính toán được lấy cảm hứng từ cấu trúc và chức năng của bộ não con người. Mạng nơ-ron truyền thẳng nhiều lớp (Multi-Layer Perceptron – MLP) gồm một lớp đầu vào, một hay nhiều lớp ẩn và một lớp đầu ra. Mỗi nơ-ron nhận các đầu vào, nhân với trọng số, cộng độ lệch (bias) rồi đưa qua hàm kích hoạt.

2.1.2. Cấu trúc mô hình

- Lớp đầu vào (Input): nhận vector đặc trưng của một người (24 đặc trưng sau mã hoá).
- Lớp ẩn (Dense): các nơ-ron kết nối đầy đủ với lớp trước, dùng hàm kích hoạt Relu.
- Lớp đầu ra (Dense + Softmax): 7 nơ-ron, mỗi nơ-ron cho xác suất của một mức cân nặng.
- Hàm kích hoạt Relu: f(x) = max(0, x), giúp mạng học được quan hệ phi tuyến và huấn luyện nhanh.
- Hàm Softmax: chuyển đầu ra thành các xác suất dương có tổng bằng 1.

2.1.3. Quá trình xây dựng mô hình

- Lan truyền thẳng: tính đầu ra từ đầu vào qua các lớp.
- Tính hàm mất mát: dùng sparse_categorical_crossentropy cho bài toán nhiều lớp với nhãn là số nguyên.
- Lan truyền ngược (backpropagation): tính gradient của mất mát theo trọng số.
- Cập nhật trọng số bằng bộ tối ưu Adam.
- Lặp lại qua nhiều epoch; dùng EarlyStopping để dừng khi mất mát trên tập validation không giảm nữa, tránh overfitting.

2.1.4. Ưu điểm

- Học được quan hệ phi tuyến phức tạp giữa các thói quen và mức cân nặng.
- Cho ra xác suất của từng lớp, thuận tiện hiển thị trên web.
- Dễ mở rộng cấu trúc và dễ lưu thành file .h5 để triển khai.

2.1.5. Nhược điểm

- Khó giải thích vì hoạt động như "hộp đen".
- Nhạy cảm với thang đo dữ liệu nên cần chuẩn hoá.
- Cần chọn nhiều siêu tham số (số lớp, số nơ-ron, batch size...).
- Có thể overfitting nếu huấn luyện quá lâu.

2.1.6. Đánh giá hiệu suất

Dùng Accuracy, Precision, Recall, F1-score cho từng lớp và ma trận nhầm lẫn. Với bài toán 7 lớp có thứ tự, các lớp liền kề (ví dụ thừa cân độ I và độ II) dễ bị nhầm lẫn với nhau.

2.2. Các kỹ thuật tiền xử lý

2.2.1. Mã hoá One-Hot

Biến cột chữ (Gender, CAEC, CALC, MTRANS) thành các cột 0/1, mỗi giá trị là một cột. Ví dụ MTRANS thành 5 cột. Ưu điểm là mô hình không hiểu nhầm có quan hệ thứ tự giữa các giá trị; nhược điểm là làm tăng số cột.

2.2.2. Chuẩn hoá StandardScaler

Biến đổi mỗi đặc trưng về trung bình 0 và độ lệch chuẩn 1: $z = (x - \mu) / \sigma$. Việc này giúp các đặc trưng có thang đo khác nhau (tuổi, số bữa ăn...) đóng góp đều cho mạng, huấn luyện ổn định và nhanh hơn. Scaler chỉ fit trên tập train rồi transform cho tập test để tránh rò rỉ dữ liệu.

2.2.3. Chia tập train/test có phân tầng (stratify)

Giữ nguyên tỷ lệ các lớp trong cả tập huấn luyện và tập kiểm tra, giúp đánh giá công bằng.

CHƯƠNG 3: GIẢI PHÁP, CÀI ĐẶT THỰC HIỆN

3.1 Mã nguồn tiền xử lý dữ liệu

- Đoạn mã này import các thư viện cần thiết và nạp dữ liệu vào chương trình:
  + pandas, numpy: xử lý và tính toán trên dữ liệu dạng bảng.
  + seaborn, matplotlib.pyplot: vẽ biểu đồ trực quan hoá.
  + pd.read_csv("obesity_level.csv"): đọc file dữ liệu thành DataFrame df.
  + df.head(): hiển thị 5 hàng đầu tiên để kiểm tra dữ liệu đã được đọc đúng.
  + df.shape, df.info(), df.describe(): cho biết kích thước, kiểu dữ liệu và thống kê mô tả của từng cột.

```
import os, warnings
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
warnings.filterwarnings("ignore")

os.makedirs("hinh", exist_ok=True)
sns.set_style("darkgrid")

ORDER = ['Insufficient_Weight', 'Normal_Weight', 'Overweight_Level_I', 'Overweight_Level_II',
         'Obesity_Type_I', 'Obesity_Type_II', 'Obesity_Type_III']

df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/obesity_level.csv")
df.head()
```

Hình 3.1. Import thư viện, đọc và hiển thị 5 hàng đầu của DataFrame

- Kết quả df.shape là (20758, 17); df.info() cho thấy 17 cột gồm 4 cột kiểu chữ (Gender, CAEC, CALC, MTRANS), nhãn NObeyesdad cũng là chữ, còn lại là số.

	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	FCVC	NCP	CAEC	SMOKE	CH2O	SCC	FAF	TUE	CALC	MTRANS	NObeyesdad
0	Male	24.443011	1.699998	81.669950	1	1	2	3	Sometimes	0	3	0	0	1	Sometimes	Public_Transportation	Overweight_Level_II
1	Female	18.000000	1.560000	57.000000	1	1	2	3	Frequently	0	2	0	1	1	no	Automobile	Normal_Weight
2	Female	18.000000	1.711460	50.165754	1	1	2	1	Sometimes	0	2	0	1	2	no	Public_Transportation	Insufficient_Weight
3	Female	20.952737	1.710730	131.274851	1	1	3	3	Sometimes	0	2	0	1	1	Sometimes	Public_Transportation	Obesity_Type_III
4	Male	31.641081	1.914186	93.798055	1	1	3	2	Sometimes	0	2	0	2	1	Sometimes	Public_Transportation	Overweight_Level_II

Hình 3.2. Kết quả hiển thị 5 hàng đầu của DataFrame

Mỗi hàng là một người với đầy đủ 17 thuộc tính. Ví dụ hàng đầu tiên là nam, khoảng 24 tuổi, cao 1,70 m, nặng khoảng 81,7 kg, thuộc mức Overweight_Level_II. Các cột Age, Height, Weight có phần thập phân dài (ví dụ 24,443011), cho thấy một phần dữ

liệu có thể được sinh thêm bằng phương pháp nội suy chứ không hoàn toàn là số liệu khảo sát gốc.

3.2 Mã nguồn chức năng làm sạch dữ liệu

3.2.1. Xử lý giá trị dữ liệu thiếu

- Câu lệnh tính phần trăm giá trị thiếu của từng cột.
  + data.isna(): Xác định các giá trị thiếu (NaN) trong data và trả về DataFrame với giá trị True/False.
  + .sum(): Tính tổng số True (giá trị thiếu) cho mỗi cột.
  + .len(data): Số lượng hàng trong data.
  + *100: Chuyển đổi tỷ lệ sang phần trăm.

```
print((df.isna().sum() / len(df)) * 100)
```

Hình 3.3. Mã nguồn xử lý giá trị thiếu

- Kết quả: tất cả 17 cột đều có tỷ lệ thiếu bằng 0%. Vì vậy nhóm không cần điền giá trị hay xoá hàng nào; khác với bài toán mẫu phải xử lý cột BMI bị thiếu.

```
Gender                            0.0
Age                               0.0
Height                            0.0
Weight                            0.0
family_history_with_overweight    0.0
FAVC                              0.0
FCVC                              0.0
NCP                               0.0
CAEC                              0.0
SMOKE                             0.0
CH2O                              0.0
SCC                               0.0
FAF                               0.0
TUE                               0.0
CALC                              0.0
MTRANS                            0.0
NObeyesdad                        0.0
dtype: float64
```

Hình 3.4. Hiển thị phần trăm giá trị thiếu của từng cột

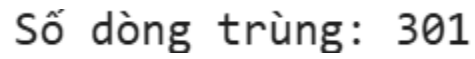
3.2.2. Kiểm tra dòng trùng lặp

- Câu lệnh df.duplicated().sum() đếm số dòng có toàn bộ giá trị giống hệt một dòng đã xuất hiện trước đó.

```
print("Số dòng trùng:", df.duplicated().sum())
```

Hình 3.5. Mã nguồn hiển thị tổng số dòng trùng lặp trong dữ liệu

- Kết quả: có 301 dòng trùng, chiếm khoảng 1,45% tổng số 20.758 dòng. Nhóm giữ nguyên các dòng này vì tỷ lệ rất nhỏ và các dòng trùng có thể là những người khác nhau nhưng có cùng thói quen sống.



Hình 3.6. Kết quả của số dòng trùng

3.2.3. Kiểm tra ngoại lai

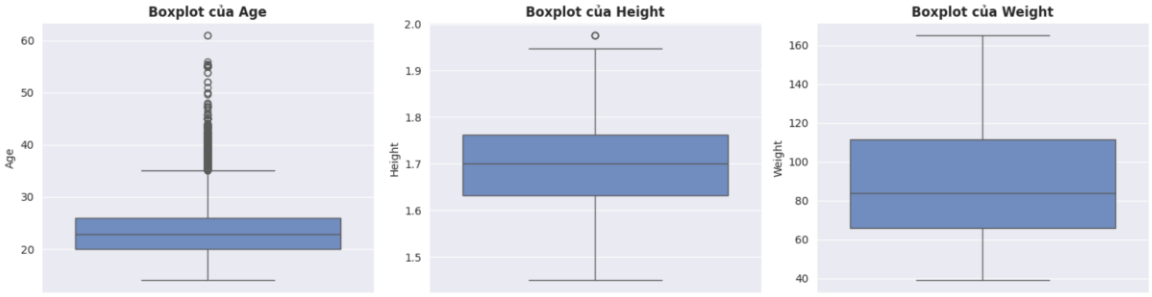
- Đoạn mã vẽ boxplot cho ba cột số liên tục là Age, Height và Weight:
  + plt.subplots(1, 3): tạo 3 biểu đồ nằm cạnh nhau, mỗi cột một biểu đồ riêng vì thang đo khác nhau.
  + sns.boxplot(y=df[col]): vẽ hộp thể hiện trung vị, tứ phân vị và các điểm ngoại lai (nằm ngoài khoảng Q1 − 1,5·IQR đến Q3 + 1,5·IQR).

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, col in zip(axes, ['Age', 'Height', 'Weight']):
    sns.boxplot(y=df[col], ax=ax, color='#5b8bd0')
    ax.set_title(f"Boxplot của {col}", fontweight='bold')
plt.tight_layout()
plt.savefig("hinh/boxplot.png", dpi=150)
plt.show()
```

Hình 3.7. Mã nguồn kiểm tra ngoại lai

- Kết quả:
  + Age: trung vị khoảng 23 tuổi, có nhiều điểm ngoại lai phía trên khoảng 35 tuổi vì dữ liệu tập trung ở người trẻ, chỉ khoảng 5% mẫu trên 35 tuổi.
  + Height: chỉ có một điểm ngoại lai nhỏ ở khoảng 1,98 m.
  + Weight: không có ngoại lai, giá trị trải từ 39 kg đến khoảng 165 kg.

Các giá trị này đều có thể xảy ra ngoài thực tế (người lớn tuổi hơn, người cao, người béo phì nặng) nên nhóm không xoá ngoại lai, để tránh mất những trường hợp thuộc nhóm béo phì cần dự đoán.



Hình 3.8. Kết quả kiểm tra ngoại lai

3.2.4. Ma trận tương quan

- Biểu đồ Heatmap thể hiện mối quan hệ tương quan giữa các cột số:
  + d['NObeyesdad_code']: mã hoá nhãn thành số 0 đến 6 theo thứ tự mức cân nặng tăng dần để tính được tương quan.
  + num_cols: danh sách các cột cần tính tương quan.
  + sns.heatmap(..., annot=True, fmt=".2f"): vẽ ma trận tương quan, hiển thị giá trị đến 2 chữ số thập phân.
  + cmap="coolwarm": màu đỏ là tương quan dương, màu xanh là tương quan âm.

```
d = df.copy()
d['NObeyesdad_code'] = d['NObeyesdad'].map({n: i for i, n in enumerate(ORDER)})
num_cols = ['Age', 'Height', 'Weight', 'family_history_with_overweight', 'FAVC', 'FCVC',
            'NCP', 'SMOKE', 'CH2O', 'SCC', 'FAF', 'TUE', 'NObeyesdad_code']

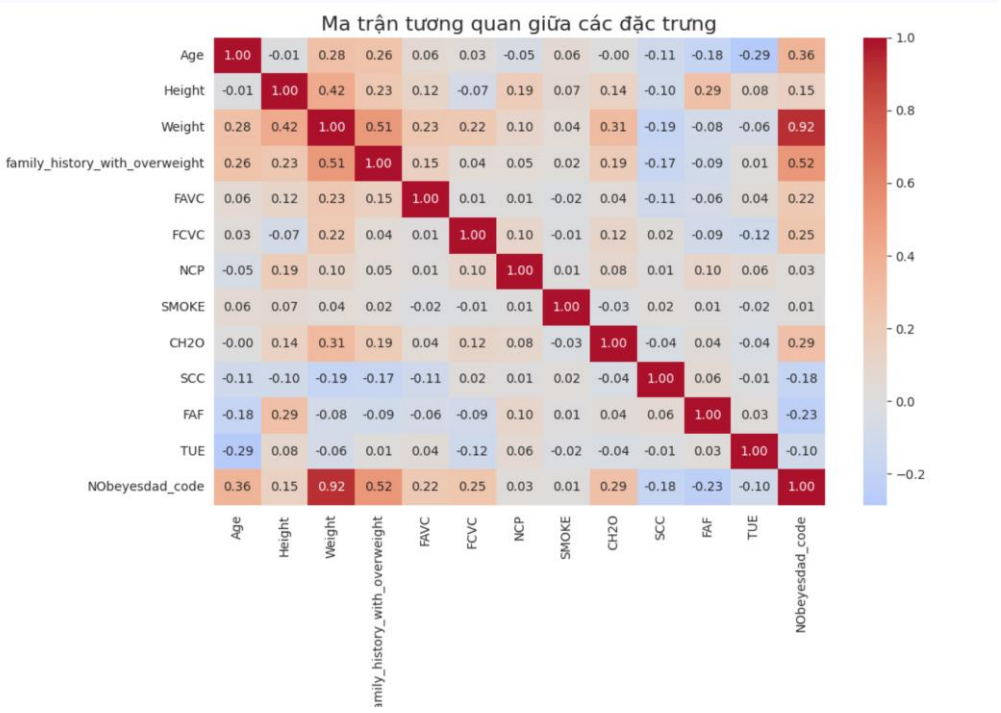
plt.figure(figsize=(11, 8))
sns.heatmap(d[num_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Ma trận tương quan giữa các đặc trưng", fontsize=16)
plt.tight_layout()
plt.savefig("hinh/heatmap.png", dpi=150)
plt.show()
```

Hình 3.9. Heatmap tương quan giữa các đặc trưng số

- Kết quả (xét tương quan với mức cân nặng):
  + Weight (0,92): tương quan rất mạnh vì mức cân nặng được xác định từ cân nặng và chiều cao. Đây là lý do nhóm bỏ cột Height và Weight khi huấn luyện.
  + family_history_with_overweight (0,52): gia đình có người thừa cân thì nguy cơ cao hơn rõ rệt.
  + Age (0,36), CH2O (0,29), FCVC (0,25), FAVC (0,22): tương quan dương ở mức yếu đến vừa.

+ FAF (−0,23), SCC (−0,18): tương quan âm, tức vận động nhiều và có theo dõi calo thì mức cân nặng thường thấp hơn.
+ SMOKE (0,01), NCP (0,03): gần như không liên quan.

Các thói quen riêng lẻ đều chỉ tương quan yếu với mức cân nặng, nên mô hình cần kết hợp nhiều đặc trưng cùng lúc.



Hình 3.10 Kết quả ma trận tương quan

3.3 Mã nguồn chức năng trực quan hóa dữ liệu

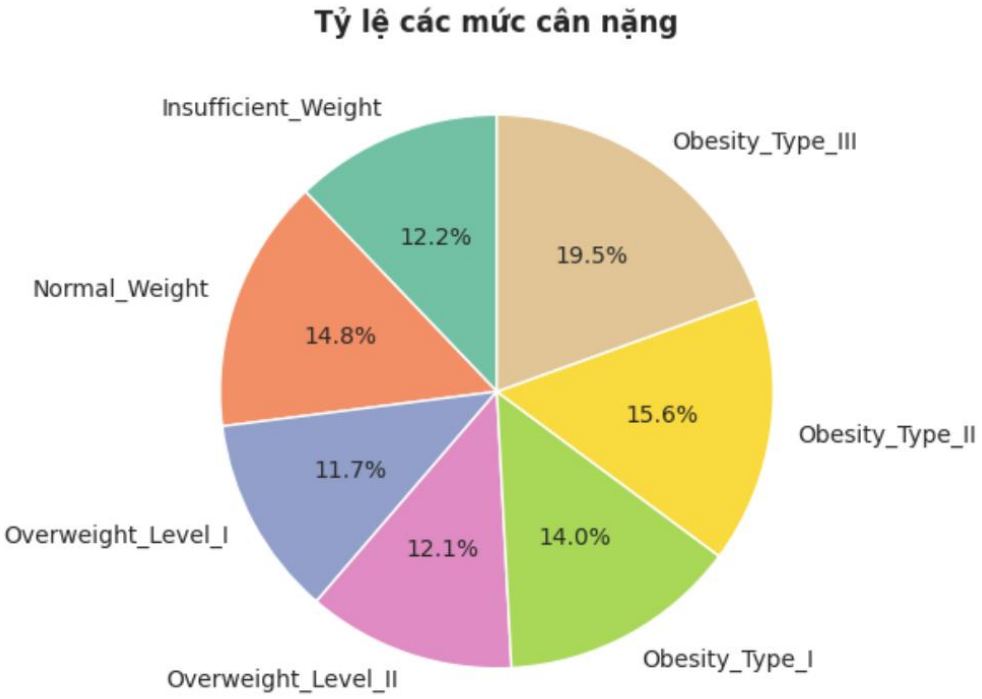
3.3.1. Phân phối 7 mức cân nặng

- ORDER: danh sách 7 mức cân nặng theo thứ tự tăng dần để các cột xếp đúng thứ tự.
- sns.countplot(...): đếm số mẫu của mỗi mức và vẽ biểu đồ cột.
- ax.annotate(...): ghi số lượng trên đầu mỗi cột.

```
# Biểu đồ tròn
counts = df['NObeyesdad'].value_counts().reindex(ORDER)
plt.figure(figsize=(6, 6))
plt.pie(counts, labels=ORDER, autopct='%1.1f%%', startangle=90,
        colors=sns.color_palette("Set2", 7))
plt.title("Tỷ lệ các mức cân nặng", fontweight='bold')
plt.tight_layout()
plt.savefig("hinh/phan_phoi_nhan_pie.png", dpi=150)
plt.show()
```

Hình 3.11. Mã nguồn phân phối 7 mức cân nặng

- Kết quả: các lớp khá cân bằng. Nhiều nhất là Obesity_Type_III (4.046 mẫu, 19,5%), ít nhất là Overweight_Level_I (2.427 mẫu, 11,7%). Vì không có lớp nào quá hiếm nên nhóm không cần dùng SMOTE.



Hình 3.12. Kết quả phân phối 7 mức cân nặng

3.3.2. Phân phối các đặc trưng liên tục

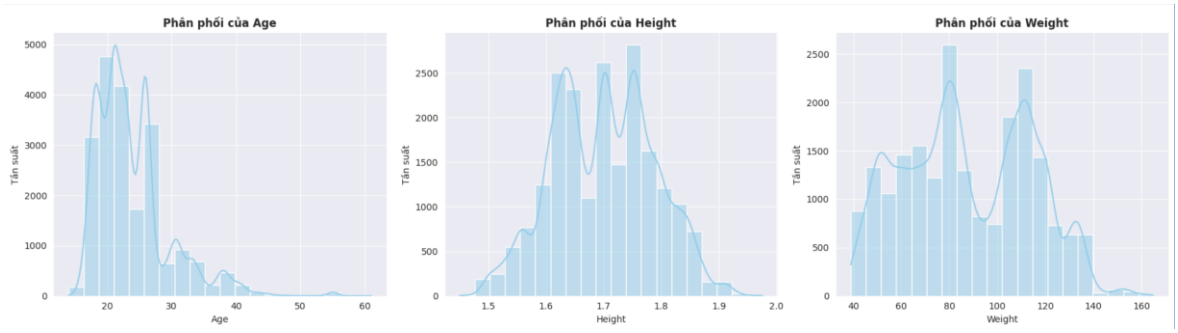
- plt.subplots(1, 3): tạo 3 biểu đồ cho Age, Height, Weight.
- sns.histplot(..., kde=True, bins=20): vẽ histogram kèm đường mật độ KDE.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, col in zip(axes, ['Age', 'Height', 'Weight']):
    sns.histplot(df[col], kde=True, ax=ax, color='skyblue', bins=20)
    ax.set_title(f"Phân phối của {col}", fontweight='bold')
    ax.set_xlabel(col); ax.set_ylabel("Tần suất")
plt.tight_layout()
plt.savefig("hinh/lien_tuc.png", dpi=150)
plt.show()
```

Hình 3.13. Mã nguồn phân phối các đặc trưng liên tục

- Kết quả:
  + **Age**: lệch phải, tập trung nhiều nhất ở khoảng 18 đến 27 tuổi, rất ít người trên 40.
  + **Height**: gần dạng chuông, tập trung quanh 1,6 đến 1,8 m.

+ **Weight**: trải rộng từ 40 đến 165 kg với nhiều đỉnh (khoảng 50, 80, 110 kg), phản ánh việc dữ liệu gồm nhiều nhóm mức cân nặng khác nhau.



Hình 3.14. Kết quả phân phối các đặc trưng liên tục

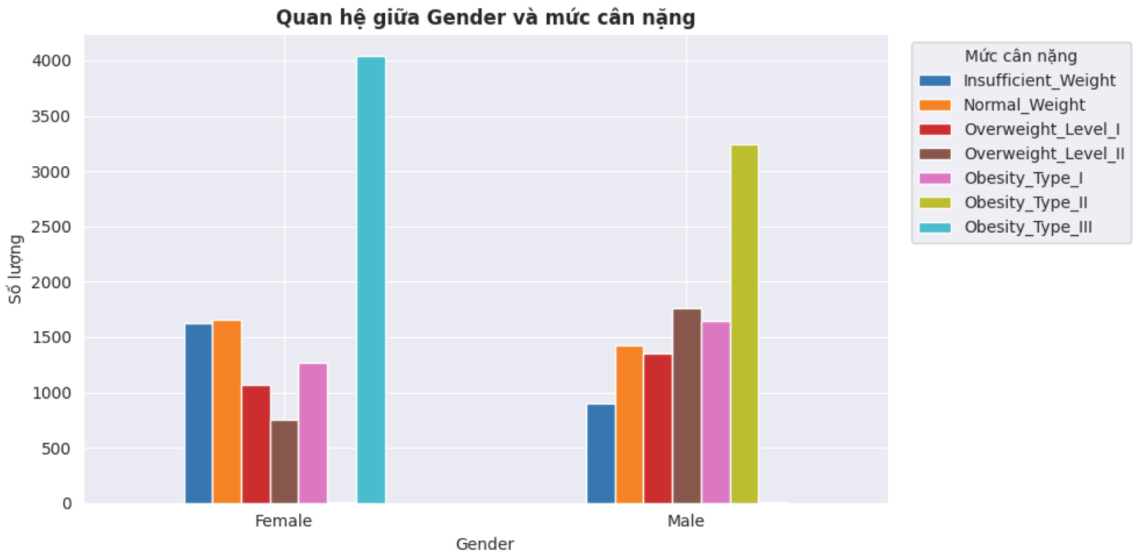
3.3.3. Quan hệ giữa thói quen và mức cân nặng

- cols: danh sách các đặc trưng cần xem xét (Gender, FAVC, FAF, CH2O, MTRANS, CAEC).
- pd.crosstab(df[col], df[‘NObeyesdad’])[ORDER]: đếm số người theo từng giá trị của thói quen và từng mức cân nặng.
- .plot(kind='bar'): vẽ biểu đồ cột nhóm, mỗi màu là một mức cân nặng.

```
cols = ['Gender', 'FAVC', 'FAF', 'CH2O', 'MTRANS', 'CAEC']
for col in cols:
    ct = pd.crosstab(df[col], df['NObeyesdad'])[ORDER]
    ct.plot(kind='bar', figsize=(10, 5), colormap='tab10')
    plt.title(f"Quan hệ giữa {col} và mức cân nặng", fontweight='bold')
    plt.xlabel(col); plt.ylabel("Số lượng")
    plt.xticks(rotation=0)
    plt.legend(title="Mức cân nặng", bbox_to_anchor=(1.02, 1), loc='upper left')
    plt.tight_layout()
    plt.savefig(f"hinh/quan_he_{col}.png", dpi=150)
    plt.show()
```

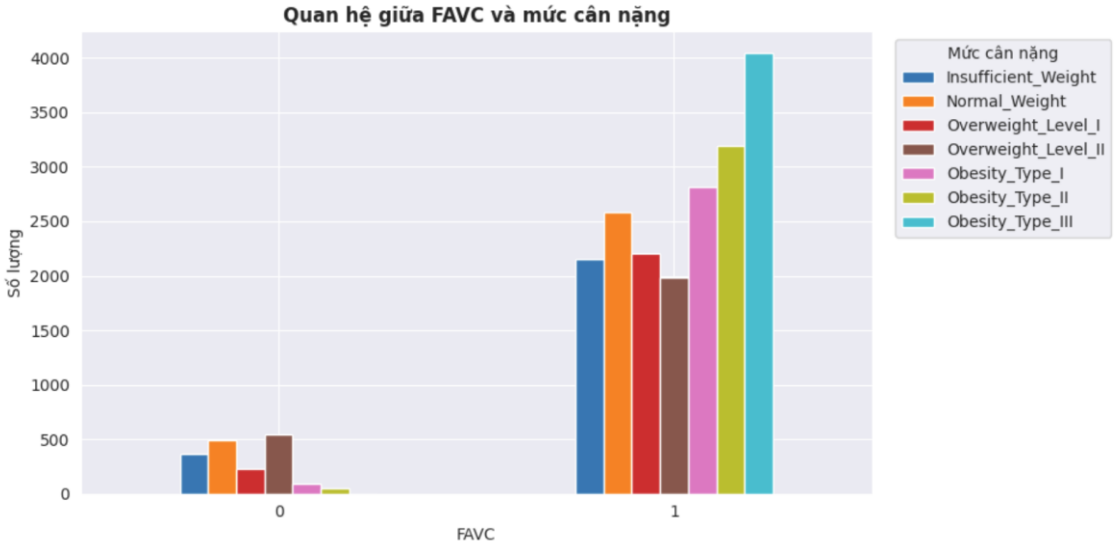
Hình 3.15. Mã nguồn quan hệ giữa thói quen và mức cân nặng

- Kết quả quan hệ giữa Gender và mức cân nặng: giới tính phân bố rất lệch giữa các mức. Obesity_Type_III gần như toàn là nữ (4.041 nữ so với 5 nam), trong khi Obesity_Type_II gần như toàn là nam (khoảng 99,8%). Đây là đặc điểm của bộ dữ liệu, không phản ánh thực tế ngoài đời, và cũng góp phần giải thích vì sao mô hình nhận diện hai nhóm này rất tốt.



Hình 3.16. Quan hệ giữa Gender và mức cân nặng

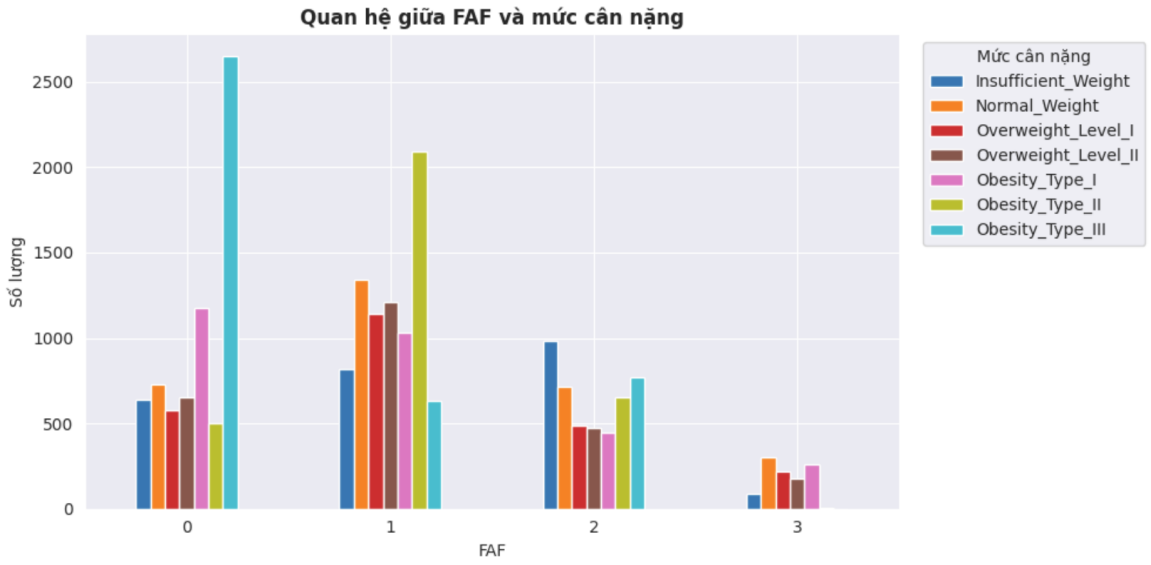
- Kết quả quan hệ giữa FAVC và mức cân nặng: nhóm thường xuyên ăn đồ nhiều calo (FAVC = 1) chiếm đa số ở mọi mức, nhưng tỷ lệ này tăng dần theo mức béo phì: khoảng 84 đến 85% ở nhóm Normal_Weight và Insufficient_Weight, 96,8% ở Obesity_Type_I, 98,3% ở Obesity_Type_II và 100% ở Obesity_Type_III.



Hình 3.17. Quan hệ giữa FAVC và mức cân nặng

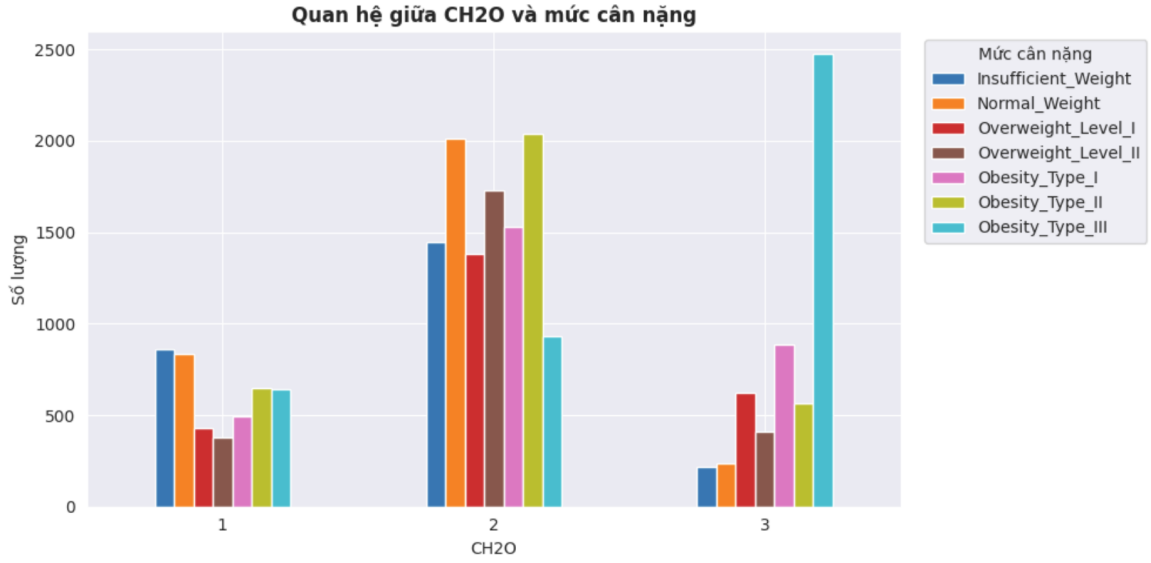
Kết quả của quan hệ giữa FAF và mức cân nặng: nhóm không vận động (FAF = 0) chiếm tỷ lệ cao ở Obesity_Type_III (khoảng 65%) và Obesity_Type_I (khoảng 40%). Ngược lại, nhóm vận động từ 5 ngày/tuần trở lên (FAF = 3) rất ít và hầu như không

có ở nhóm béo phì độ II, III. Điều này phù hợp với tương quan âm của FAF với mức cân nặng ở Hình 3.10.



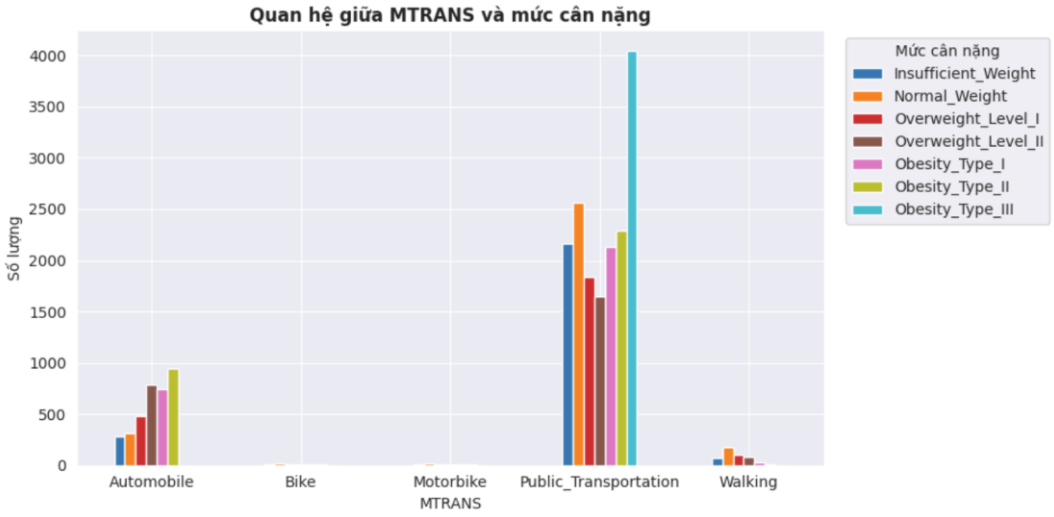
Hình 3.18. Quan hệ giữa FAF và mức cân nặng

- Kết quả của quan hệ giữa CH20 và mức cân nặng: tỷ lệ người uống trên 2 lít nước mỗi ngày (CH2O = 3) cao nhất ở Obesity_Type_III (khoảng 61%) và thấp ở Insufficient_Weight, Normal_Weight (khoảng 8%). Đây là mối liên hệ nên đọc thận trọng: uống nhiều nước không gây béo phì, mà có thể do cách dữ liệu được sinh ra hoặc do người béo phì thường được khuyên uống nhiều nước hơn.



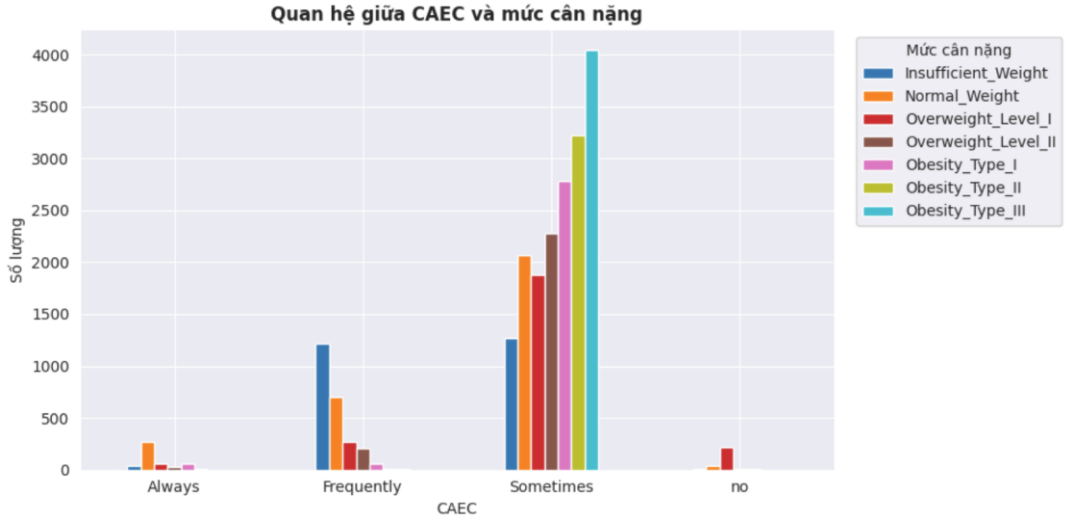
Hình 3.19. Quan hệ giữa CH2O và mức cân nặng

- Kết quả của quan hệ giữa MTRANS và mức cân nặng: phương tiện công cộng (Public_Transportation) chiếm đa số ở mọi mức (từ 65% đến 100%). Các phương tiện Motorbike, Bike, Walking có rất ít mẫu (lần lượt 38, 32 và 467 mẫu trên toàn bộ dữ liệu), nên mô hình khó học được ảnh hưởng của các nhóm này. Tỷ lệ dùng ô tô cao hơn ở các nhóm thừa cân và béo phì độ I, II (khoảng 20 đến 31%).



Hình 3.20. Quan hệ giữa MTRANS và mức cân nặng

- Kết quả của quan hệ giữa CAEC và mức cân nặng: thói quen ăn vặt giữa các bữa có xu hướng rõ rệt. Nhóm "Thường xuyên" (Frequently) chiếm 48% ở Insufficient_Weight nhưng giảm dần còn dưới 2% ở Obesity_Type_I, II, III; ngược lại nhóm "Thỉnh thoảng" (Sometimes) tăng dần từ 50% lên gần 100% ở các nhóm béo phì.



Hình 3.21. Quan hệ giữa CAEC và mức cân nặng

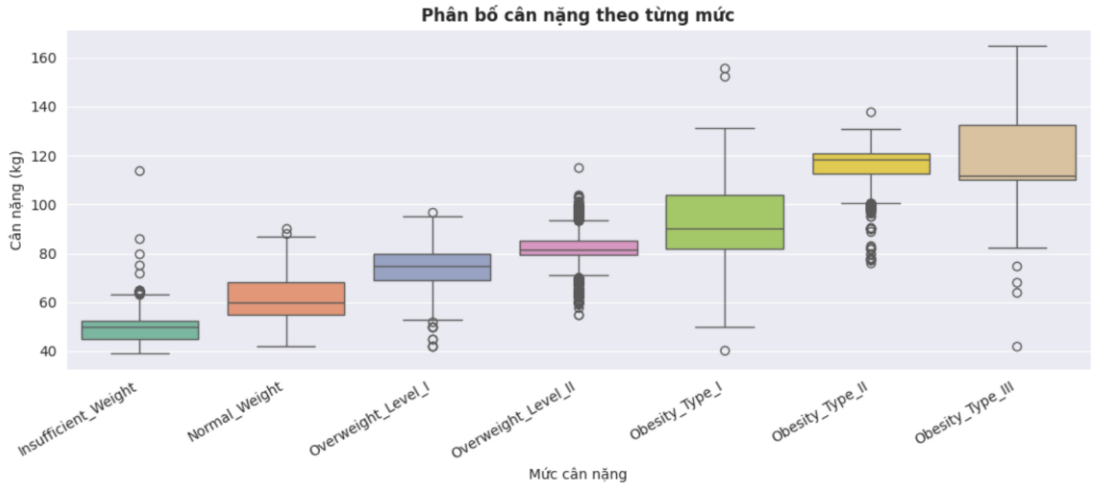
3.3.4. Phân bố Weight theo từng mức cân nặng

- sns.boxplot(data=df, x='NObeyesdad', y='Weight', order=ORDER): vẽ boxplot cân nặng cho từng mức theo đúng thứ tự.
- plt.xticks(rotation=30): xoay nhãn trục x cho dễ đọc.

```
plt.figure(figsize=(11, 5))
sns.boxplot(data=df, x='NObeyesdad', y='Weight', order=ORDER, palette="Set2")
plt.title("Phân bố cân nặng theo từng mức", fontweight='bold')
plt.xlabel("Mức cân nặng"); plt.ylabel("Cân nặng (kg)")
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.savefig("hinh/weight_theo_nhan.png", dpi=150)
plt.show()
```

Hình 3.22. Mã nguồn phân bố Weight theo từng hạng cân

- Kết quả: cân nặng trung vị tăng dần theo mức: khoảng 50 kg (Insufficient_Weight), 60 kg (Normal_Weight), 75 kg (Overweight_Level_I), 82 kg (Overweight_Level_II), 90 kg (Obesity_Type_I), 118 kg (Obesity_Type_II) và 112 kg (Obesity_Type_III). Các hộp của những mức liền kề chồng lấn lên nhau (đặc biệt Normal, Overweight_Level_I và Overweight_Level_II) vì mức cân nặng còn phụ thuộc chiều cao; điều này cho thấy chỉ dựa vào cân nặng cũng không tách được hoàn toàn các mức.



Hình 3.23. Mức cân nặng theo từng hạng cân

3.4. Chuyển đổi dữ liệu

3.4.1. Mã hoá nhãn và xử lý dữ liệu chuỗi

- X = df.iloc[:, :-1]: lấy toàn bộ cột đặc trưng (16 cột đầu).

- y = df.iloc[:, -1].map({...}): lấy cột nhãn và đổi 7 mức cân nặng thành số nguyên từ 0 đến 6 theo ORDER.
- X.drop(columns=['Height', 'Weight']): bỏ hai cột chiều cao và cân nặng.
- pd.get_dummies(X, dtype=float): mã hoá One-Hot các cột chữ (Gender, CAEC, CALC, MTRANS) thành các cột 0/1.
- feature_names: lưu lại danh sách tên cột sau mã hoá để web dùng khi dự đoán.

```
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.iloc[:, :-1]
y = df.iloc[:, -1].map({n: i for i, n in enumerate(ORDER)}).astype(int)

X = X.drop(columns=['Height', 'Weight'])      # chỉ dự đoán từ thói quen
X = pd.get_dummies(X, dtype=float)            # One-Hot
feature_names = list(X.columns)
print("Số đặc trưng sau mã hoá:", len(feature_names))
X.head()
```

Hình 3.24. Mã nguồn mã hoá nhãn và xử lý dữ liệu chuỗi

- Kết quả: sau mã hoá, dữ liệu còn **24 đặc trưng** (10 cột số, Gender 2 cột, CAEC 4 cột, CALC 3 cột, MTRANS 5 cột).

Số đặc trưng sau mã hoá: 24

	Age	family_history_with_overweight	FAVC	FCVC	NCP	SMOKE	CH2O	SCC	FAF	TUE	...	CAEC_Sometimes	CAEC_no	CALC_Frequently	CALC_Sometimes	CALC_no	MTRANS_Automobile	MTRANS_Bike	MTRANS_Mo
0	24.443011	1	1	2	3	0	3	0	0	1	...	1.0	0.0	0.0	1.0	0.0	0.0	0.0	
1	18.000000	1	1	2	3	0	2	0	1	1	...	0.0	0.0	0.0	0.0	1.0	1.0	0.0	
2	18.000000	1	1	2	1	0	2	0	1	2	...	1.0	0.0	0.0	0.0	1.0	0.0	0.0	
3	20.952737	1	1	3	3	0	2	0	1	1	...	1.0	0.0	0.0	1.0	0.0	0.0	0.0	
4	31.641081	1	1	3	2	0	2	0	2	1	...	1.0	0.0	0.0	1.0	0.0	0.0	0.0	

Hình 3.25. Mã hoá nhãn và xử lý dữ liệu chuỗi

Nhóm cố ý bỏ hai cột Height và Weight. Mức cân nặng được xác định từ chỉ số BMI = cân nặng / chiều cao², nên nếu giữ hai cột này mô hình gần như chỉ tính lại BMI và không còn ý nghĩa dự đoán từ thói quen sống. Sau mã hoá, dữ liệu có 24 đặc trưng: 10 cột số, Gender 2 cột, CAEC 4 cột, CALC 3 cột, MTRANS 5 cột.

3.4.2. Chia dữ liệu và chuẩn hoá

- train_test_split(..., test_size=0.15, random_state=42, stratify=y): chia 85% dữ liệu để huấn luyện và 15% để kiểm tra; stratify=y giữ nguyên tỷ lệ các lớp ở hai tập; random_state=42 giúp kết quả lặp lại được.

- StandardScaler(): chuẩn hoá mỗi đặc trưng về trung bình 0 và độ lệch chuẩn 1.
- fit_transform trên tập train, transform trên tập test: chỉ học tham số chuẩn hoá từ tập train để tránh rò rỉ thông tin từ tập test.
- joblib.dump(...): lưu scaler.pkl và feature_names.pkl để dùng lại trong ứng dụng web.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
pd.DataFrame(X_train, columns=feature_names).head()
```

Hình 3.26. Mã nguồn chia dữ liệu và chuẩn hoá

- Kết quả: tập huấn luyện có 17.644 mẫu, tập kiểm tra có 3.114 mẫu, mỗi mẫu gồm 24 đặc trưng.

```
(17644, 24) (17644,)
(3114, 24) (3114,)
```

	Age	family_history_with_overweight	FAVC	FCVC	NCP	SMOKE	CH2O	SCC	FAF	TUE	...	CAEC_Sometimes	CAEC_no	CALC_Frequently	CALC_Sometimes	CALC_no	MTRA
0	-0.868418	0.470866	0.307254	0.970463	0.328794	-0.108154	1.384526	-0.18354	0.020519	0.575584	...	0.430066	-0.116184	-0.16416	0.612110	-0.571678	
1	0.554097	0.470866	0.307254	0.970463	0.328794	-0.108154	-1.549133	-0.18354	-1.132466	-0.949511	...	-2.325227	-0.116184	-0.16416	0.612110	-0.571678	
2	-1.027943	0.470866	0.307254	0.970463	-2.434362	-0.108154	-0.082303	-0.18354	0.020519	2.100679	...	0.430066	-0.116184	-0.16416	-1.633693	1.749236	
3	1.187831	0.470866	0.307254	-0.794052	0.328794	-0.108154	-1.549133	-0.18354	1.173504	0.575584	...	0.430066	-0.116184	-0.16416	0.612110	-0.571678	
4	-0.092760	0.470866	0.307254	0.970463	0.328794	-0.108154	-0.082303	-0.18354	1.173504	-0.949511	...	0.430066	-0.116184	-0.16416	0.612110	-0.571678	

5 rows × 24 columns

Hình 3.27. Kết quả chia dữ liệu và chuẩn hoá

3.4.3. Về dữ liệu mất cân bằng

Dữ liệu này khá cân bằng (từ 11,7% đến 19,5% mỗi lớp) nên không cần SMOTE. Nhóm chỉ dùng stratify=y khi chia dữ liệu để giữ tỷ lệ lớp.

3.5. Huấn luyện mô hình

3.5.1. Xây dựng mạng nơ-ron

- Sequential([...]): xếp các lớp nối tiếp nhau.
- Input(shape=(24,)): lớp đầu vào nhận 24 đặc trưng.
- Dense(64, activation='relu') và Dense(32, activation='relu'): hai lớp ẩn dùng hàm kích hoạt ReLU.

- Dense(7, activation='softmax'): lớp đầu ra cho xác suất của 7 mức cân nặng.
- compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=[‘accuracy’]): dùng hàm mất mát cho nhãn là số nguyên, bộ tối ưu Adam và đo độ chính xác.

```
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(len(ORDER), activation='softmax'),
])
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

Hình 3.28. Mã nguồn xây dựng mạng Nơ ron

- Kết quả model.summary(): lớp dense có 1.600 tham số, dense_1 có 2.080 tham số và dense_2 có 231 tham số, tổng cộng 3.911 tham số (khoảng 15,28 KB), tất cả đều là tham số huấn luyện được. Đây là mô hình nhỏ, nhẹ, phù hợp với dữ liệu dạng bảng và chạy nhanh khi dự đoán trên web.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,600
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 7)	231

Total params: 3,911 (15.28 KB)
Trainable params: 3,911 (15.28 KB)
Non-trainable params: 0 (0.00 B)

Hình 3.29. Cấu trúc mô hình

3.5.2. Huấn luyện và xuất file .h5

- EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True): dừng huấn luyện khi val_loss không giảm sau 10 epoch liên tiếp và khôi phục trọng số tốt nhất, giúp tránh overfitting.
- model.fit(..., epochs=100, batch_size=32, validation_split=0.2): huấn luyện tối đa 100 epoch, mỗi lần lấy 32 mẫu, dùng 20% tập train làm tập validation.
- history: lưu lại loss và accuracy qua từng epoch để vẽ đồ thị.
- model.save(‘modelNN.h5’): xuất mô hình đã huấn luyện ra file .h5.

```
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=10, restore_best_weights=True)

history = model.fit(X_train, y_train, epochs=100, batch_size=32,
                    validation_split=0.2, callbacks=[early_stop], verbose=1)

model.save('modelNN.h5')
```

Hình 3.30. Mã nguồn huấn luyện và xuất file .h5

Nhóm dùng 20% tập train làm tập validation (khoảng 14.115 mẫu huấn luyện, 3.529 mẫu validation). EarlyStopping dừng khi val_loss không cải thiện sau 10 epoch và khôi phục trọng số tốt nhất. Mô hình được lưu ra modelNN.h5 (khoảng 80 KB).

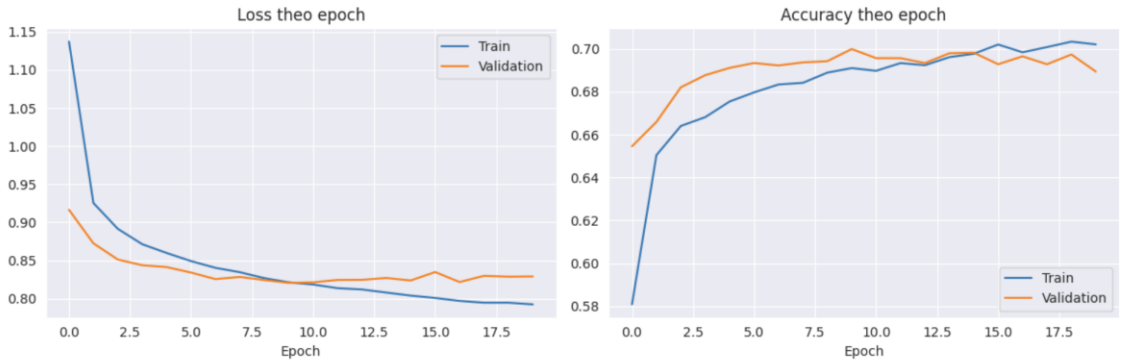
- Đoạn mã dùng history.history để vẽ hai đồ thị cạnh nhau: loss và accuracy của tập train (màu xanh) và validation (màu cam) theo từng epoch.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(history.history['loss'], label='Train')
axes[0].plot(history.history['val_loss'], label='Validation')
axes[0].set_title("Loss theo epoch"); axes[0].set_xlabel("Epoch"); axes[0].legend()

axes[1].plot(history.history['accuracy'], label='Train')
axes[1].plot(history.history['val_accuracy'], label='Validation')
axes[1].set_title("Accuracy theo epoch"); axes[1].set_xlabel("Epoch"); axes[1].legend()
plt.tight_layout()
plt.savefig("hinh/loss_acc.png", dpi=150)
plt.show()
```

Hình 3.31. Mã nguồn vẽ đồ thị loss và accuracy theo epoch

- Kết quả:
  + Loss: giảm nhanh ở các epoch đầu (từ khoảng 1,14 xuống dưới 0,9 ở tập train), sau đó giảm chậm dần. Loss validation đi ngang ở khoảng 0,82 từ epoch thứ 10 trở đi trong khi loss train vẫn giảm, nghĩa là mô hình bắt đầu học thuộc dữ liệu huấn luyện.
  + Accuracy: tăng từ khoảng 0,58 (train) lên khoảng 0,70; accuracy validation đạt khoảng 0,69 đến 0,70.
  + Huấn luyện dừng sớm sau khoảng 20 epoch nhờ EarlyStopping. Hai đường train và validation sát nhau nên mô hình không bị overfitting nặng.



Hình 3.32. Đồ thị Loss và Accuracy

3.5.3. Đánh giá mô hình

- model.predict(X_test): dự đoán xác suất 7 lớp cho từng mẫu trong tập kiểm tra.
- .argmax(axis=1): chọn lớp có xác suất cao nhất làm kết quả dự đoán.
- accuracy_score và classification_report: tính độ chính xác tổng thể và Precision, Recall, F1-score cho từng lớp.

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

proba = model.predict(X_test, verbose=0)
pred = proba.argmax(axis=1)

print("Test Accuracy:", round(accuracy_score(y_test, pred), 4))
print(classification_report(y_test, pred, target_names=ORDER, digits=3))
```

Hình 3.33. Mã nguồn Đánh giá mô hình

- Nhận xét:
  + Độ chính xác khoảng 67,7% trên bài toán 7 lớp, cao hơn nhiều so với đoán ngẫu nhiên (khoảng 14%), trong khi mô hình chỉ dùng thói quen sống mà không dùng chiều cao, cân nặng.
  + Nhóm Obesity_Type_III (F1 = 0,981) và Obesity_Type_II (F1 = 0,832) được nhận diện rất tốt. Một phần nguyên nhân là hai nhóm này có đặc điểm rất riêng trong dữ liệu (gần như toàn nữ và toàn nam như Hình 3.16), nên mô hình có thể đang tận dụng đặc điểm của bộ dữ liệu.
  + Các mức trung gian Overweight_Level_I (F1 = 0,439), Overweight_Level_II (F1 = 0,422) và Normal_Weight (F1 = 0,517) cho kết quả thấp nhất. Recall của hai mức thừa cân chỉ khoảng 0,37 và 0,39, tức mô hình bỏ sót khoảng 60% số ca ở các mức này. Nguyên nhân là ranh

giới giữa các mức liền kề rất mờ và thói quen sống của họ gần giống nhau (xem Hình 3.23).

+ Kết quả này thấp hơn so với khi dùng thêm chiều cao và cân nặng, nhưng phù hợp với mục tiêu của đề tài là dự đoán từ thói quen sống.

Test Accuracy: 0.6769

	precision	recall	f1-score	support
Insufficient_Weight	0.650	0.741	0.693	379
Normal_Weight	0.520	0.515	0.517	462
Overweight_Level_I	0.538	0.371	0.439	364
Overweight_Level_II	0.457	0.392	0.422	378
Obesity_Type_I	0.580	0.595	0.588	437
Obesity_Type_II	0.762	0.916	0.832	487
Obesity_Type_III	0.974	0.988	0.981	607
accuracy			0.677	3114
macro avg	0.640	0.645	0.639	3114
weighted avg	0.665	0.677	0.667	3114

Hình 3.34 Kết quả đánh giá mô hình

CHƯƠNG 4: XÂY DỰNG ỨNG DỤNG

Trong bài toán này, sử dụng Flask (Python) để đưa mô hình lên giao diện web chạy cục bộ, toàn bộ giao diện bằng tiếng Việt, không cần dịch vụ bên ngoài như ngrok.

- Cấu trúc thư mục:

```
obesity_web/
├── app.py              # web Flask
├── train.py            # huấn luyện, xuất modelNN.h5
├── modelNN.h5          # mô hình đã huấn luyện
├── scaler.pkl          # bộ chuẩn hoá
├── feature_names.pkl   # danh sách 24 đặc trưng
├── obesity_level.csv   # dữ liệu
├── requirements.txt
├── templates/          # base.html, gioi_thieu.html, du_doan.html
└── static/style.css
```

Hình 4.1. Cấu trúc thư mục

- Các trang của ứng dụng:
  + Trang Giới thiệu (/): giới thiệu cách hoạt động (3 bước: nhập thông tin, mô hình tính toán, xem kết quả) và liệt kê 7 mức cân nặng kèm mô tả.
  + Trang Dự đoán (/du-doan): biểu mẫu nhập thông tin chia 3 nhóm:

    Thông tin cơ bản: giới tính, tuổi, gia đình có người thừa cân.

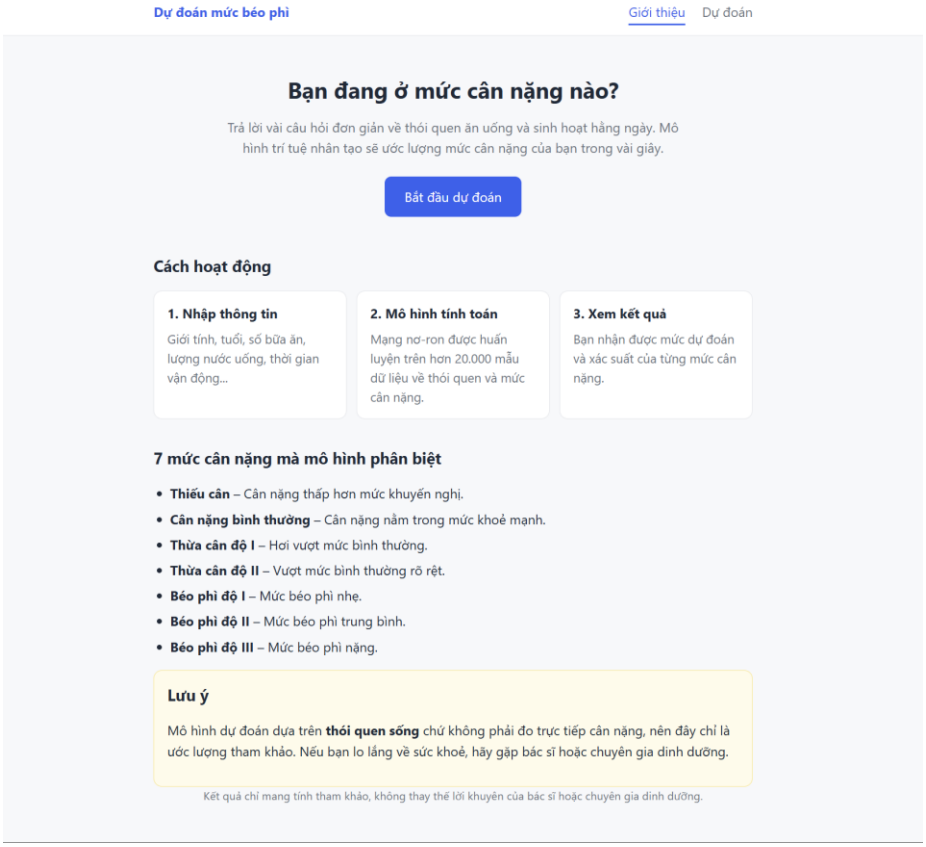
    Thói quen ăn uống: ăn đồ nhiều calo, mức độ ăn rau, số bữa chính, ăn vặt, lượng nước uống (theo lít), theo dõi calo, uống rượu bia.

    Vận động và sinh hoạt: tần suất vận động, thời gian dùng thiết bị, phương tiện di chuyển, hút thuốc.

  Các lựa chọn dùng đơn vị dễ hiểu như "Dưới 1 lít", "3 bữa", "1 đến 2 ngày mỗi tuần".
- Luồng xử lý khi người dùng bấm "Dự đoán":
  + doc_form() đọc và kiểm tra dữ liệu nhập (đúng kiểu, nằm trong khoảng hợp lệ); nếu sai thì báo lỗi bằng tiếng Việt.
  + du_doan() đổi các ô yes/no thành 1/0, One-Hot, sắp xếp cột đúng theo feature_names.pkl, chuẩn hoá bằng scaler.pkl.
  + Mô hình modelNN.h5 trả về xác suất của 7 lớp; lớp có xác suất cao nhất là kết quả.

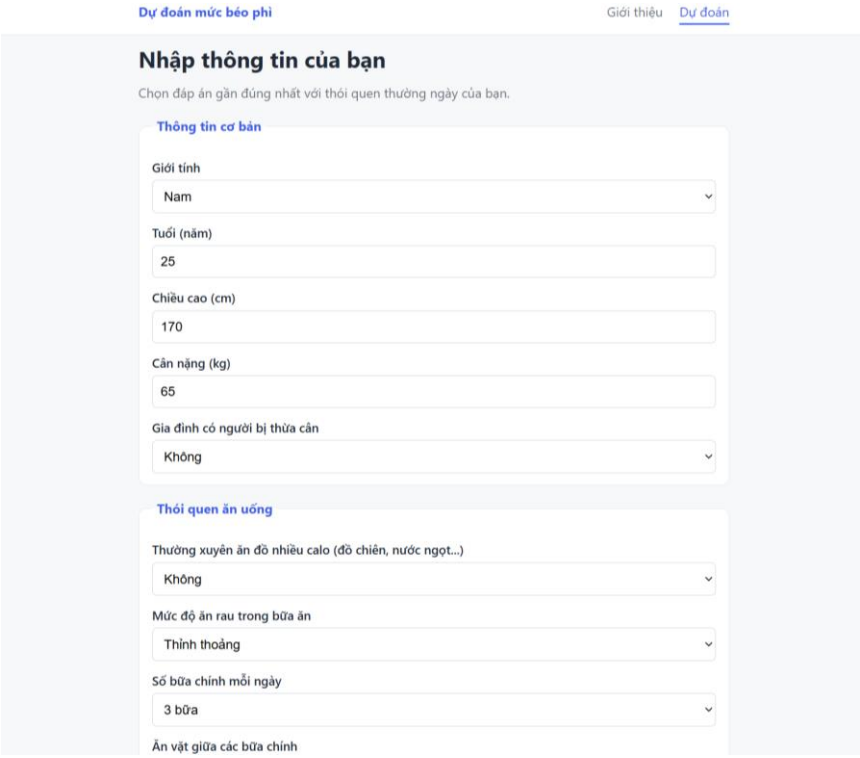
+ Trang hiển thị tên mức cân nặng bằng tiếng Việt, mô tả ngắn và thanh phần trăm của cả 7 mức.

- Trang giới thiệu gồm các phần:
  + Thanh điều hướng ở đầu trang có tên ứng dụng “Dự đoán mức béo phì” và hai mục Giới thiệu, Dự đoán; mục đang mở được gạch chân.
  + Phần mở đầu với câu hỏi “Bạn đang ở mức cân nặng nào?”, một đoạn giải thích ngắn và nút “Bắt đầu dự đoán” để chuyển sang trang nhập thông tin.
  + Cách hoạt động gồm 3 thẻ: (1) Nhập thông tin, (2) Mô hình tính toán (mạng nơ-ron được huấn luyện trên hơn 20.000 mẫu), (3) Xem kết quả (mức dự đoán và xác suất từng mức).
  + 7 mức cân nặng mà mô hình phân biệt, từ Thiếu cân đến Béo phì độ III, mỗi mức có một dòng mô tả ngắn.
  + Khung “Lưu ý” nền vàng nhắc rằng mô hình dự đoán dựa trên thói quen sống chứ không đo trực tiếp cân nặng, nên chỉ là ước lượng tham khảo; nếu lo lắng về sức khỏe cần gặp bác sĩ hoặc chuyên gia dinh dưỡng.



Hình 4.2. Trang giới thiệu

- Trang Dự đoán là một biểu mẫu chia thành 3 nhóm, mỗi nhóm nằm trong một khung riêng:
  + Thông tin cơ bản: giới tính, tuổi (năm) và "Gia đình có người bị thừa cân".
  + Thói quen ăn uống: ăn đồ nhiều calo, mức độ ăn rau, số bữa chính mỗi ngày, ăn vặt giữa các bữa, lượng nước uống mỗi ngày, có theo dõi calo hay không và uống rượu bia.
  + Vận động và sinh hoạt: tần suất tập thể dục, thời gian dùng điện thoại/máy tính/TV, phương tiện di chuyển chính và hút thuốc.



Hình 4.3. Trang nhập thông tin

Các lựa chọn được viết bằng ngôn ngữ đời thường như "Dưới 1 lít", "3 bữa", "1 đến 2 ngày mỗi tuần" thay cho các mã số trong dữ liệu gốc, giúp người dùng không cần hiểu kỹ thuật vẫn nhập được. Ô tuổi chỉ nhận số hợp lệ; nếu nhập sai, hệ thống hiện thông báo lỗi để người dùng sửa lại. Cuối trang có nút "Dự đoán".

Luồng xử lý khi người dùng bấm "Dự đoán":

+ doc_form() đọc và kiểm tra dữ liệu nhập (đúng kiểu, nằm trong khoảng cho phép).
+ du_doan() đổi các ô yes/no thành 1/0, mã hoá One-Hot, sắp xếp cột đúng theo feature_names.pkl rồi chuẩn hoá bằng scaler.pkl.

+ Mô hình modelNN.h5 trả về xác suất của 7 mức; mức có xác suất cao nhất là kết quả.

- Sau khi dự đoán, trang tự cuộn xuống khung kết quả gồm:
  + Mức cân nặng dự đoán hiển thị chữ lớn, kèm một dòng giải thích ý nghĩa (ví dụ "Cân nặng bình thường – Cân nặng nằm trong mức khoẻ mạnh").
  + Xác suất của cả 7 mức dưới dạng thanh ngang và phần trăm; mức được chọn được in đậm và tô sáng. Ví dụ trong hình, mô hình cho "Cân nặng bình thường" 99,3% và "Thừa cân độ I" 0,6%, các mức còn lại gần bằng 0%.
  + Dòng lưu ý ở cuối: đây chỉ là ước lượng tham khảo từ thói quen sống, không phải chẩn đoán y khoa.



Hình 4.4. Kết quả dự đoán

Việc hiển thị xác suất của cả 7 mức giúp người dùng biết mô hình chắc chắn đến đâu. Nếu hai mức liền kề có xác suất gần nhau thì kết quả kém chắc chắn hơn, phù hợp với nhận xét ở Chương 3 rằng các mức trung gian dễ bị nhầm lẫn.

KẾT LUẬN

1. Kết quả đạt được

a) Về kiến thức

- Hiểu bài toán phân loại đa lớp và cách hoạt động của mạng nơ-ron nhiều lớp: lan truyền thẳng, hàm kích hoạt ReLU, Softmax, hàm mất mát cross-entropy, bộ tối ưu Adam và Early Stopping.
- Nắm được các kỹ thuật tiền xử lý: kiểm tra giá trị thiếu, dòng trùng, ngoại lai, mã hóa One-Hot, chuẩn hóa StandardScaler, chia dữ liệu có phân tầng.
- Hiểu các chỉ số đánh giá mô hình phân loại: Accuracy, Precision, Recall, F1-score và ma trận nhầm lẫn.
- Biết cách đưa mô hình học máy vào ứng dụng web bằng Flask: nạp mô hình, kiểm tra dữ liệu nhập và hiển thị kết quả.
- Sử dụng các thư viện Pandas, Scikit-learn, TensorFlow/Keras, Joblib, Matplotlib và Seaborn.

b) Về sản phẩm

- Phân tích, làm sạch và trực quan hóa bộ dữ liệu 20.758 mẫu.
- Xây dựng và huấn luyện mạng nơ-ron 24 đặc trưng đầu vào, 7 lớp đầu ra, đạt độ chính xác khoảng 68% trên tập kiểm tra khi chỉ dùng thói quen sống; nhận diện tốt béo phì độ III (F1 khoảng 0,98) và độ II (khoảng 0,83).
- Xuất mô hình ra file modelNN.h5 cùng scaler.pkl, feature_names.pkl.
- Xây dựng website Flask hoàn toàn bằng tiếng Việt gồm trang giới thiệu và trang dự đoán: biểu mẫu 14 ô nhập chia 3 nhóm, kiểm tra dữ liệu nhập, hiển thị mức cân nặng dự đoán kèm xác suất của cả 7 mức.

2. Những hạn chế của đề tài

- Các mức trung gian (thừa cân độ I, độ II, cân nặng bình thường) còn bị nhầm lẫn nhiều.
- Dữ liệu dựa trên khảo sát tự khai báo nên có thể thiếu chính xác; một số nhóm ít mẫu (xe máy, xe đạp, đi bộ) khiến mô hình khó học.
- Mô hình chỉ dùng thói quen nên không thể thay thế việc đo BMI thực tế.

- Chưa so sánh với các thuật toán khác và chưa tinh chỉnh siêu tham số một cách hệ thống.

3. Hướng phát triển của đề tài

Để khắc phục các hạn chế và nâng cao chất lượng sản phẩm, các hướng phát triển được đề xuất như sau:

- Thử thêm các mô hình khác: Random Forest, XGBoost, SVM và dùng GridSearchCV để so sánh, tinh chỉnh.
- Bổ sung chế độ dùng thêm chiều cao, cân nặng (đặt DUNG_CHIEU_CAO_CAN_NANG = True trong train.py) để dự đoán chính xác hơn.
- Dùng SHAP hoặc LIME để giải thích thói quen nào ảnh hưởng nhiều nhất đến kết quả.
- Thu thập thêm dữ liệu thực, nhất là người lớn tuổi và các nhóm ít mẫu.
- Đưa website lên máy chủ hoặc làm ứng dụng di động, lưu lịch sử dự đoán và gợi ý thay đổi thói quen cho từng người.

TÀI LIỆU THAM KHẢO

[1] TS. Hoàng Quốc Việt. Bài giảng Học máy. Trường Đại học Sư phạm Kỹ thuật Hưng Yên.

[2] TS. Nguyễn Văn Hậu, TS. Nguyễn Duy Tân, ThS. Nguyễn Thị Hải Năng, ThS. Nguyễn Hoàng Điệp. Giáo trình Python cơ bản. Nhà xuất bản Đại học Quốc gia Hà Nội.

[3] Huang, Shih-Chia, and Trung-Hieu Le. Principles and labs for deep learning. Academic Press, 2021.

[4] https://machinelearningcoban.com/